安卓 OTA 库接口使用文档

1、导入和使用 OTA 库

导入：

将 libwsota.arr 库拷贝到 Android Studio app/libs 目录中，并在 build.gradle 中添加：

```
repositories {
    flatDir {
        dirs 'libs'
    }
}

dependencies {
    compile(name: 'libwsota', ext: 'aar')
    implementation fileTree(dir: 'libs', include: ['*.jar'])
    implementation 'androidx.appcompat:appcompat:1.2.0'
    implementation 'androidx.constraintlayout:constraintlayout:1.1.3'
```

使用：

类名称为：WSOTA，直接使用并根据 Android Studio 提示导入即可。

```
import com.ws.libwsota.WSOTA;
```

2、OTA 接口及说明

1）加载固件

接口名：WSOTA.load(byte[] firmware)

参数：frimware 类型为 byte[]，是 OTA 固件读取后的字节数组。

返回值（String）：null：表示成功

errMsg：错误提示字符串

示例：

```
String errMsg = WSOTA.load(frimData);
if (errMsg != null) {
    showDialog( title: "固件加载失败", errMsg);
    return;
}
```

2）获取固件信息

接口名：WSOTA.getFrimwareInfo()

使用条件：固件加载成功之后

返回值（WSOTA.FrimInfo）：

null：固件信息不存在

非 null：固件信息

WSOTA.FrimInfo 成员定义为：

```
///固件信息对象
public static class FrimInfo {
    public int frimType = -1;
    public int frimVer = -1;
    public int frimProductId = -1;
    public int frimVendorId = -1;
    public int frimSize = -1;
    public boolean frimEncrypt = false;
```

frimType：固件类型（范围：0-65535，0：应用固件，1：协议栈）

frimVer：固件版本（范围：0-65535）

frimProductId：固件产品 ID（范围：0-65535），固件对应产品的 ID

frimVendorId：固件厂商 ID（范围：0-65535），固件对应产品的厂商 ID

frimSize：固件大小（单位：字节）

frimEncrypt：固件内容加密性（false：未加密，true：安全加密）

示例代码：

```
942        /* 更新固件信息 */
943        WSOTA.FrimInfo frimInfo = WSOTA.getFrimwareInfo();
944        if (frimInfo != null) {
945            encryptTypeTextView.setText(frimInfo.frimEncrypt ? "AES安全" : "无");
946            productIdTextView.setText(String.format("%04X", frimInfo.frimProductId));
947            frimwareTypeTextView.setText(frimInfo.frimType == 0 ? "应用" :
948                    frimInfo.frimType == 1 ? "协议栈" :
949                            String.format("%04X", frimInfo.frimType));
950            frimwareVersionTextView.setText(String.format("%04X", frimInfo.frimVer));
951            frimwareSizeTextView.setText(frimInfo.frimSize + " B");
952        }
```

3）注册 OTA 事件

接口名：WSOTA.setEventCallback(WSOTA. EventCallback callback)

说明：注册事件回调可监听和处理 OTA 的各种事件，下面是各类事件回调的说明。

方法：otaDeviceScanCallback

线程上下文：**非主线程回调**，不可在回调中直接操作界面，需要使用 Handler 机制

说明：在调用 WSOTA.scanStart 成功后，扫描到蓝牙设备时触发回调。

处理示例：

```
@Override
public void otaDeviceScanCallback(WSOTA.DeviceInfo deviceInfo, BluetoothDevice device, int rssi) {
    deviceItemAdapter.addDevice(device, rssi, deviceInfo);
}
```

方法：otaBeaconFailCallback

线程上下文：**主线程回调**，可直接操作界面

说明：调用 WSOTA. otaBeaconStart 开启失败时，触发回调。

处理示例：

```
@Override
public void otaBeaconFailCallback(String errMsg, int errCode) {
    otaBeaconStop();

    showDialog( title: "打开失败", errMsg);
}
```

方法：otaBeaconSuccessCallback

线程上下文：**主线程回调**，可直接操作界面

说明：调用 WSOTA.otaBeaconStart 成功打开时，触发回调。

处理示例：

```
@Override
public void otaBeaconSuccessCallback() {
}
```

方法：otaConnectFailCallback

线程上下文：**主线程回调**，可直接操作界面

说明：调用 WSOTA.deviceConnect 后，连接失败时，触发回调

处理示例：

```
@Override
public void otaConnectFailCallback(String errMsg, int errCode) {
    updateDeviceConnectState("disconnected");

    showDialog( title: "连接失败", errMsg);
}
```

方法：otaConnectSuccessCallback
线程上下文：**主线程回调**，可直接操作界面
说明：调用 WSOTA.deviceConnect 后，设备连接成功时回调
处理示例：

```
@Override
public void otaConnectSuccessCallback(WSOTA.DeviceInfo deviceInfo) {
    updateDeviceConnectState("connected");
    showToast( msg: "获取设备信息完成");

    updateDeviceInfo(deviceInfo);
}
```

方法：otaDeviceInfoUpdateCallback
线程上下文：**主线程回调**，可直接操作界面
说明：设备连接上时，设备信息更改时回调
处理示例：

```
@Override
public void otaDeviceInfoUpdateCallback(WSOTA.DeviceInfo deviceInfo) {
    updateDeviceInfo(deviceInfo);
}
```

方法：otaDisconnect
线程上下文：**主线程回调**，可直接操作界面
说明：在 OTA 成功连接后（otaConnectSuccessCallback 触发后），发生断开时回调
处理示例：

```
@Override
public void otaDisconnect(String errMsg) {
    updateDeviceConnectState("disconnected");
}
```

方法：otaModeSetSuccessCallback
线程上下文：**主线程回调**，可直接操作界面
说明：调用 WSOTA.rebootMode 后，操作成功时回调
处理示例：

```
@Override
public void otaModeSetSuccessCallback() {
    showToast( msg: "切换成功");
}
```

方法：otaModeSetFailCallback
线程上下文：**主线程回调**，可直接操作界面
说明：调用 WSOTA.rebootMode 后，操作失败时回调
处理示例：

```
@Override
public void otaModeSetFailCallback(String errMsg) {
    showDialog( title: "切换失败", errMsg);
}
```

方法：otaUpdateProgressCallback
线程上下文：**主线程回调**，可直接操作界面
说明：OTA 升级进度通知，当 txSize 等于 firmSize 时升级成功
处理示例：

```
@Override
public void otaUpdateProgressCallback(int firmSize, int txSize) {
    if (txSize == firmSize) {
        updateMsg( title: "更新成功，耗时：" + String.format("%.2f", (new Date().getTime() - updateStartTime) / 1000f) + "S",  msg: null);
        return;
    }

    updateProgress((float) txSize / (float) firmSize);
}
```

方法：otaUpdateFailCallback
线程上下文：**主线程回调**，可直接操作界面
说明：OTA 升级失败时，触发回调
处理示例：

```
@Override
public void otaUpdateFailCallback(String errMsg) {
    updateMsg( title: "更新失败", errMsg);
}
```

4）OTA 升级广播开启

接口名：WSOTA.otaBeaconStart()

使用条件：固件加载成功后

注：若固件重新加载，请调用 stopUpdateBeacon 停止升级广播

说明：OTA 升级广播的作用是手机向空中广播固件信息，当低版本的设备接收到符合自己产品且版本号更高的固件广播，则自动重启进入升级模式。该广播用于唤醒周围可升级的设备。成功后触发 otaBeaconSuccessCallback 回调，失败时 otaBeaconFailCallback。

示例代码：

```
/* 开启OTA Beacon*/
private void otaBeaconStart() {
    otaBeaconSwitch.setChecked(true);

    WSOTA.otaBeaconStart();
}
```

5）OTA 升级广播停止

接口名：WSOTA.otaBeaconStop()

停止 OTA 升级广播

示例代码：

```
847     /* 关闭OTA Beacon */
848     private void otaBeaconStop() {
849         otaBeaconSwitch.setChecked(false);
850
851         WSOTA.otaBeaconStop();
852     }
```

6）连接设备

接口名：WSOTA.deviceConnect(Context context, BluetoothDevice device)

使用条件：固件加载成功之后

参数：

context：Android 视图控件的上下文

BluetoothDevice：蓝牙设备（可通过扫描获取）

说明：连接一个蓝牙设备，成功后回调 otaConnectSuccessCallback，失败时回调 otaConnectFailCallback。

注：deviceConnect 返回非 null 的错误信息后，不会触发成功或失败的回调。

示例代码：

```
/* 连接设备 */
String errMsg = WSOTA.deviceConnect( context: this, currentDevice);
if (errMsg == null) {
    updateDeviceConnectState("connecting");
} else {
    updateDeviceConnectState("disconnected");
    showDialog( title: "连接失败", errMsg);
}
```

7）断开设备

接口名称：WSOTA.deviceDisconnect()

参数：无

断开当前 OTA 设备，并中止所有的操作。

示例：

```
WSOTA.deviceDisconnect();
updateDeviceConnectState("disconnected");
```

8）重启设备到指定模式

接口名称：WSOTA.rebootToMode(int mode, boolean reconnect)

参数：

mode：设备工作模式

WSOTA. WORK_MODE_NORMAL：正常模式

WSOTA. WORK_MODE_UPDATE：升级模式

WSOTA. WORK_MODE_FISSION：裂变模式

WSOTA. WORK_MODE_FORCE_FISSION：强制裂变模式

模式说明：

正常模式：设备正常工作时的状态

升级模式：该模式可**被手机或者其他设备升级**

裂变模式：该模式设备可自动为低版本的设备进行升级

强制裂变模式：该模式设备可为高版本或低版本的设备进行升级（版本不相同）

reconnect：**切换后，是否重新连接设备，不重新连接默认断开。重新连接时触发设备信息更新回调。不触发连接成功或者失败回调，连接失败后触发设备断开回调。**

注：返回非 null 的错误信息后，不会触发成功或失败的回调。

示例代码：

```
String errMsg = WSOTA.rebootToMode(mode, reconnect: true);
if (errMsg != null) {
    showDialog( title: "切换失败", errMsg);
    return;
}
```

9）启动固件更新

接口名称：WSOTA.startUpdate(int mode)

参数：

mode：更新成功后，进入指定的模式。

说明：操作成功后，将更新失败回调或持续产生更新进度回调。

注：返回非 null 的错误信息后，不会触发进度或失败的回调。

示例代码：

```
String errMsg = WSOTA.startUpdate(WSOTA.WORK_MODE_FISSION);

if (errMsg != null) {
    showDialog( title: "更新失败", errMsg);
    return;
}
```

10）更新中止

接口名：WSOTA.abortUpdate()

参数：无

说明：中止当前更新，调用默认成功，不会触发 update 任何回调

示例代码：

```
WSOTA.abortUpdate();
```

11）启动设备扫描

接口名称：WSOTA.scanStart()

参数：无

说明：启动蓝牙扫描，扫描结果通过 otaDeviceScanCallback 事件返回。

示例代码：

```
WSOTA.scanStart();
```

12）停止设备扫描

接口名称：WSOTA.scanStop()

参数：无

说明：停止扫描

示例代码：

```
WSOTA.scanStop();
```

13）解析扫描数据

接口名称：WSOTA. parseScanRecord(BluetoothDevice device, byte[] scanRecord)

参数：

device：蓝牙设备

scanRecord：扫描到的广播数据

返回值：

解析出的设备信息。

说明：该接口用于自己实现设备扫描时，解析扫描到广播数据而用。

3、OTA 工作流程

1） 加载 OTA 固件
2） 扫描蓝牙设备
3） 打开 OTA 升级广播（不是必须步骤，在 IOS 中需要切换蓝牙为从角色）（2，3 不分先后）
4） 连接设备（连接前请调用接口停止 OTA 升级广播，同时停止扫描）
5） 设备连接不成功，返回（2）
6） 连接成功后，可进行 7，8，9（不分先后）
7） 重启设备到指定模式（正常，升级，裂变，强制裂变模式）
8） 启动更新（仅设备处于正常模式或升级模式可用）
9） 断开设备